

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего
образования
«Национальный исследовательский
Нижегородский государственный университет им. Н.И. Лобачевского»
(ННГУ)

Институт информационных технологий, математики и механики

**Кафедра: Высокопроизводительных вычислений и системного
программирования**

Направление подготовки: «01.03.02 Прикладная математика и информатика»
Профиль подготовки: «Прикладная математика и информатика (Общий профиль)»

ОТЧЕТ

по научно-исследовательской практике

на тему:

**«Сравнительный анализ производительности и статистических свойств
генераторов псевдослучайных чисел»**

Выполнил: студент группы 3823Б1ПМоп3
_____ Лопатин А.А

Научный руководитель:
Профессор кафедры ВВиСП, д.т.н.
_____ Турлапов В.Е

Нижний Новгород

2026

Содержание

	Стр.
Введение	3
1 Классификация генераторов псевдослучайных чисел	4
1.1 ГПСЧ на основе модульной арифметики	5
1.2 ГПСЧ на основе бинарной арифметики.....	6
2 Оценка производительности ГПСЧ.....	11
2.1 Тестовая конфигурация	11
2.2 Алгоритм тестирования	11
2.3 Тестируемые ГПСЧ.....	12
2.4 Результаты и анализ	13
3 Оценка статистического качества ГПСЧ.....	17
3.1 Алгоритм тестирования	17
3.2 Результаты и анализ	19
Литература	22

Введение

Ключевую роль в Монте-Карло моделировании распространения света в рассеивающих средах выполняют генераторы псевдослучайных чисел (ГПСЧ). Разные алгоритмы генерации отличаются производительностью и статистическим качеством генерируемых последовательностей, влияя на скорость вычислений и точность метода [1, 2].

В данной работе приведен сравнительный анализ шести алгоритмов, представляющих разные классы псевдослучайных генераторов: `taus88`, `mt11213b`, `mt19937`, `xoshiro256+`, `xoshiro256++` и `mcg59`. Сравнивались латентность и пропускная способность генераторов. Статистическое качество оценивалось с помощью тестового пакета `PractRand` [3, 4].

1 Классификация генераторов псевдослучайных чисел

Псевдослучайные числа генерируются вычислительной машиной с помощью заданного алгоритма, использующего детерминированные функции. Каждое число, генерируемое такими функциями, полностью определяется предыдущим, что порождает неслучайные (в смысле истинной случайности) последовательности [5]. Псевдослучайность достигается за счет стохастических свойств генерируемой последовательности, длины ее периода и случайного выбора начального значения (сиды).

В общем виде функция псевдослучайной генерации представима с помощью композиции функций $g \circ f$. Функция $f: \mathbb{Z}^m \rightarrow \mathbb{Z}^m$ такова, что $\vec{x}_n = f(\vec{x}_{n-1})$, где $\vec{x}_n$ есть состояние генератора на шаге n , m – размер состояния. Полученное на шаге n состояние используется для вычисления очередного псевдослучайного числа $r_n = g(\vec{x}_n)$.

Функция $g(f(\vec{x}_{n-1}))$ определяет статистическое качество генератора. Алгоритм генерации считается качественным, если рассмотренная композиция функций $g \circ f$ выдает некоррелированные числа в краткосрочной перспективе (продолжительность определяется периодом генератора) и имитирует заданное распределение (как правило, равномерное) на дистанции [6].

Рассмотрим идеи некоторых классов ГПСЧ, представители которых были выбраны для сравнения.

1.1 ГПСЧ на основе модульной арифметики

ГПСЧ на основе модульной арифметики генерируют элементы некоторого кольца (или поля) вычетов $\mathbb{Z}/m\mathbb{Z}$ (далее – $\mathbb{Z}_m$), используя целочисленную рекуррентную формулу с операцией взятия модуля.

1. Линейный конгруэнтный генератор.

Фундаментальным классом ГПСЧ является класс линейных конгруэнтных генераторов (Linear Congruential Generator, LCG), впервые описанный в работе [7] в 1951 году.

Функция генерации LCG определяется рекуррентным соотношением $x_n = (a \cdot x_{n-1} + c) \bmod m$, где $m > 0$ и $a, c, x_0 \in \mathbb{Z}_m$ – параметры, выбор которых определяет период генератора и, вообще говоря, качество порождаемых чисел [5, с. 31-45]. Для генерации псевдослучайного числа используется тождественное преобразование $r_n = x_n$.

LCG в свое время получил широкое распространение благодаря простоте и эффективности предлагаемой реализации (как известно, взятие модуля по основанию 2 на уровне машинных вычислений реализовано быстрой операцией битового сдвига). Известным представителем данного класса является генератор `rand()` стандартной библиотеки языка C [8].

2. Мультипликативный конгруэнтный генератор.

Частным случаем LCG является случай $c = 0$, выделяющий класс мультипликативных конгруэнтных генераторов (Multiplicative Congruential Generator, MCG) вида $x_n = a \cdot x_{n-1} \bmod m$.

Согласно Д. Э. Кнуту [5, с. 30], ограничение $c = 0$ уменьшает максимальный период генератора (поскольку $x_n = 0$ теперь недостижимо в невырожденной последовательности), но увеличивает скорость

вычислений. Доказано, что максимальный период $m - 1$ достигается при условии простоты m , примитивности¹ a и взаимной простоты x_0 с m [5, с. 39-41].

В качестве представителя класса мультипликативных конгруэнтных генераторов выбран 59-битный MCG59, где $a = 13^{13}$, $m = 2^{59}$, и период равен 2^{57} (максимальный период не достигается, так как m не является простым числом) [9].

1.2 ГПСЧ на основе бинарной арифметики

ГПСЧ на основе бинарной арифметики оперируют элементами поля $GF(2)$, применяя битовые операции XOR, AND, SHIFT, ROTATE к двоичным последовательностям (векторам битов).

1. Генератор на регистрах сдвига с линейной обратной связью.

Обширный класс ГПСЧ использует для генерации псевдослучайных последовательностей аппаратно реализуемые регистры сдвига с линейной обратной связью (Linear Feedback Shift Register, LFSR) [1, с. 11-13].

Идейно LFSR есть специальный регистр, ежетапно применяющий к хранящемуся значению операцию битового сдвига и записывающий в освободившуюся ячейку результат некоторой линейной комбинации битов предыдущего состояния, где в качестве операции сложения используется XOR (исключающее ИЛИ). Выбор конкретных битов определяется

¹ Примитивность некоторого элемента a конечного поля $GF(p)$ по определению означает, что a порождает мультипликативную группу $GF(p)^*$, т.е. любой ненулевой элемент поля $GF(p)$ представим как a^k , где $k \in \mathbb{Z}$. Например, для поля $\mathbb{Z}_5$ примитивным является элемент 2, так как порождаемая им в $\mathbb{Z}_5$ циклическая группа содержит элементы $\{2^0 = 1, 2^1 = 2, 2^2 = 4, 2^3 = 3\} = \mathbb{Z}_5^*$.

многочленом обратной связи. Выходные биты генератора в простейшем случае формируются из отброшенных в результате сдвига битов LFSR.

Известным представителем данного класса является генератор `random()` стандартной библиотеки языка C [8].

Очевидно вычислительное преимущество такого рода генераторов над генераторами с модульной арифметикой: низкоуровневые операции с битовыми строками выполняются быстрее эквивалентных целочисленных. Данная вычислительная гипотеза неоднократно подтверждалась эмпирическими данными [2, 10, 11].

2. Tausworthe генератор.

Представители класса Tausworthe [12] используют для генерации рекуррентную формулу $x_n = (a_1 \cdot x_{n-1} + \dots + a_k \cdot x_{n-k}) \bmod 2$, где $k > 0, a_k = 1$ и $a_1, \dots, a_{k-1} \in GF(2)$. (1)

Не любой набор коэффициентов $a_1, \dots, a_{k-1}$ породит статистически качественный генератор. Автор алгоритма показал, что для достижения максимального периода генератора 2^{k-1} необходима примитивность² над $GF(2)$ характеристического полинома $P(z) = 1 + a_1z + a_2z^2 + \dots + z^k$ [12].

На практике для обновления состояния Tausworthe генератора используется вышеупомянутый LFSR, что обеспечивает высокую скорость

² Полином степени n над конечным полем $GF(p)$ называют примитивным, если он неприводим над $GF(p)$, и каждый его корень в $GF(p^n)$ примитивен. Например, $P(z) = z^2 + z + 1$ неприводим в $GF(2)[z]$ и в $GF(2^2) = \{0, 1, z_0, z_0 + 1\}$ имеет корень z_0 (т.е. $P(z_0) = 0$ и $(z_0)^2 = z_0 + 1$), порождающий мультипликативную группу $GF(2^2)^* = \{(z_0)^1 = z_0, (z_0)^2 = z_0 + 1, (z_0)^3 = z_0 \cdot (z_0)^2 = (z_0)^2 + z_0 = (z_0 + 1) + z_0 = 1\}$. $P(z)$ примитивен в $GF(2)[z]$.

псевдослучайной генерации. Высокое статистическое качество выходной последовательности достигается путем комбинирования нескольких независимых Tausworthe генераторов при помощи сложения (XOR) выходов их LFSR [14].

На основе данной идеи реализован генератор taus88 [13], взятый в сравнение как представитель класса комбинированных Tausworthe. Он использует результаты трех LFSR, управляемых генераторами с периодами $k_1 = 31$, $k_2 = 29$ и $k_3 = 28$, что дает общий период длиной $p = (2^{31} - 1)(2^{29} - 1)(2^{28} - 1) \approx 2^{88}$ при размере состояния 12 байт [14].

3. Mersenne Twister.

Широко известный класс генераторов Mersenne Twister [15] развивает идею LFSR, предлагая использовать для генерации обобщение LFSR со скручиванием (twisted generalization of LFSR, TGFSR) [16, 17].

Некоторые k битов выходной последовательности LFSR образуют k -битное слово вида $X_n = x_{j_1+n-1}x_{j_2+n-1}\dots x_{j_k+n-1}$, где x_i получен по формуле (*), а $j_1, \dots, j_k$ есть т.н. отступы, задающие шаблон выбора битов. Данные последовательности используются для вычисления ω -битных строк $X_{l+n} = X_{l+m} \oplus X_l A$ ($l = 0, 1, \dots$), где A – матрица $\omega \times \omega$ специального вида с элементами из $GF(2)$, n, m, ω – положительные целые и $n > m$.

Генераторы типа Mersenne Twister (MT) дополнительно перемешивают битовые строки для генерации состояния, применяя формулу $X_{k+n} = X_{k+m} \oplus (X_k^u | X_{k+1}^l)A$, где $|$ есть операция конкатенации векторов, сцепляющая (в некотором упрощении) u старших битов строки X_k и l младших битов строки X_{k+1} (точный алгоритм описан в оригинальной статье [15]). Полученное на шаге n состояние X_n

дополнительно умножается на матрицу выходного преобразования B над $GF(2)$, размер которой согласуется с X_n и определяется параметрами генератора. Так генерируется результирующее слово $Y_n = BX_n$.

Стоит отметить, что на уровне вычислений весь алгоритм реализуется при помощи двоичных операций с битовыми строками, а матричная форма записи эквивалентна последовательности битовых преобразований, упакованной в матрицы A и B .

Классическим примером Mersenne Twister генератора является MT19937, имеющий как 32-битную, так и 64-битную версии и предоставляемый стандартной библиотекой C++ [18]. Он имеет период $p = 2^{19937} - 1$ с размером состояния 2496 байт [17]. Его STL-реализация взята в сравнение вместе с Boost-реализацией генератора MT11213b [19].

4. Xorshift генератор.

Наиболее быстрые (среди генераторов, проходящих TestU01³ [20]) скалярные CPU генераторы с эталонным статистическим качеством принадлежат классу Xorshift [10, 21].

Основная идея генераторов данного класса заключается в использовании нескольких битовых слов-состояний для генерации результата посредством серии битовых сдвигов и операций XOR. В алгоритмах отдельных представителей класса дополнительно используются операции битовой ротации и умножения на константу [22, 23].

В общем случае Xorshift генератор представим рекуррентным соотношением $X_n = AX_{n-1}$, где A - матрица специального вида,

³ TestU01 является универсальным тестовым пакетом для оценки статистического качества генераторов псевдослучайных чисел. Включает три тестовых батареи: Small Crush, Crush и Big Crush (106 тестов). Эталоном считается генератор, проходящий все тесты Big Crush без систематических ошибок.

называемая матрицей перехода. Умножение на данную матрицу эквивалентно серии битовых операций сдвига и XOR. Вид матрицы A определяет статистические свойства генератора.

Результат генерации Y_n , как и в случае с Mersenne Twister, вычисляется умножением на матрицу выходного преобразования B , вид которой определяется спецификой класса [21, 22].

Для сравнения выбраны 64-битные генераторы xoshiro256+ [24] и xoshiro256++ [25] с периодами $p = 2^{256} - 1$ и размерами состояния 32 байта.

2 Оценка производительности ГПСЧ

В данном разделе описывается экспериментальное окружение и методология бенчмаркинга, приводятся результаты тестирования.

2.1 Тестовая конфигурация

Производительность измерялась на ПК с процессором AMD Ryzen 5 3500X (@3.60 ГГц, Zen 2), памятью 32 Гб DDR4 (@3600 МГц) и ОС Windows 11 Home (25H2). Программа написана на языке C++ и скомпилирована с помощью gcc (13.3.0) с ключами -O3 -DNDEBUG -fno-unroll-loops -fno-exceptions -fno-rtti. Компиляция и запуск тестов производились в окружении WSL2 с ОС Ubuntu 24.04.4 LTS.

2.2 Алгоритм тестирования

Тесты производительности реализованы средствами библиотеки Google Benchmark [26]. Латентность генератора измерялась как время, затрачиваемое тестируемой функцией на генерацию одного случайного числа (нс/вызов). Для замера пропускной способности (млн.чисел/с) генератор заполнял псевдослучайными числами буфер (**std::vector**) размера **N = 1e6**. Согласно результатам исследования [2] пропускная способность генераторов увеличивается с ростом размера буфера и стабилизируется после 1e5 элементов для большинства CPU генераторов. В тестах пропускной способности после вычислительного цикла дополнительно устанавливался барьер **benchmark::ClobberMemory()**. Для функций генерации случайного числа использовался инлайнинг.

В целях обеспечения единообразия оптимизаций для всех генераторов флаг **-march=native** намеренно не использовался. Анализ отчета компилятора показал, что ни один из вычислительных циклов векторизован не был. Во избежание различного поведения компилятора в отношении

циклов, соответствующих разным ГПСЧ, также использовался флаг `-fno-unroll-loops`.

Запуск тестов осуществлялся с помощью bash-скрипта **autorun.sh**, который устанавливал процессу высокий приоритет (`nice -20`), привязывал процесс к фиксированному ядру и задавал параметры бенчмарка. Для каждого генератора бенчмарк выполнялся по 12 раз на сценарий с предварительным прогревом кэша в течение 0.1 секунды. Тесты запускались в случайном порядке с автоматической рандомизацией. Согласно данным [27] такая конфигурация бенчмарка минимизирует выборочное стандартное отклонение времени и улучшает воспроизводимость результатов. Частота процессора фиксировалась на отметке в 3.60 ГГц. На время тестирования отключались алгоритмы рандомизации адресного пространства ОС (ASLR) и использования пространства подкачки (swap).

В качестве результата бенчмарка выбиралась медиана общего времени (**real_time**). Усреднение пропускной способности осуществлялось средствами Google Benchmark с помощью метода **SetItemsProcessed()**. Учитывалось 3 знака после запятой с округлением вверх последнего разряда. Относительная скорость вычислялась по значениям пропускной способности.

2.3 Тестируемые ГПСЧ

Для тестирования выбраны 6 генераторов: `taus88`, `mt11213b`, `mt19937`, `xoshiro256+`, `xoshiro256++` и `mcg59`. Использовались реализации `taus88` и `mt11213b` из библиотеки `Boost.Random:x64 (1.90.0)` [28], `stl`-реализация `mt19937` [18], оригинальные авторские реализации `xoshiro256+` [24] и `xoshiro256++` [25] и классическая реализация `mcg59`.

Список исследуемых ГПСЧ представлен в таблице 1.

№	Генератор	Период	Размер состояния, байт	Примечание
1	taus88	2^{88}	12	Boost.Random
2	mt11213b	$2^{11213} - 1$	1404	Boost.Random
3	mt19937	$2^{19937} - 1$	2496	C++ STL
4	xoshiro256+	$2^{256} - 1$	32	xoshiro256plus.hpp
5	xoshiro256++	$2^{256} - 1$	32	xoshiro256plusplus.hpp
6	mcg59	2^{57}	8	mcml_mcg59.h

Таблица 1. Тестируемые ГПСЧ

2.4 Результаты и анализ

В таблице 2 и на рисунках 1, 2 представлены результаты тестирования латентности и пропускной способности ГПСЧ.

№	Генератор	Латентность, нс/вызов	ПС, млн.чисел/с	Относительная скорость
1	taus88	1.845	506.875	47%
2	mt11213b	2.841	580.341	54%
3	mt19937	3.081	505.050	47%
4	xoshiro256+	0.720	1080.695	100%
5	xoshiro256++	0.876	874.197	81%
6	mcg59	1.144	869.918	80%

Таблица 2. Результаты бенчмарка (медианные значения)

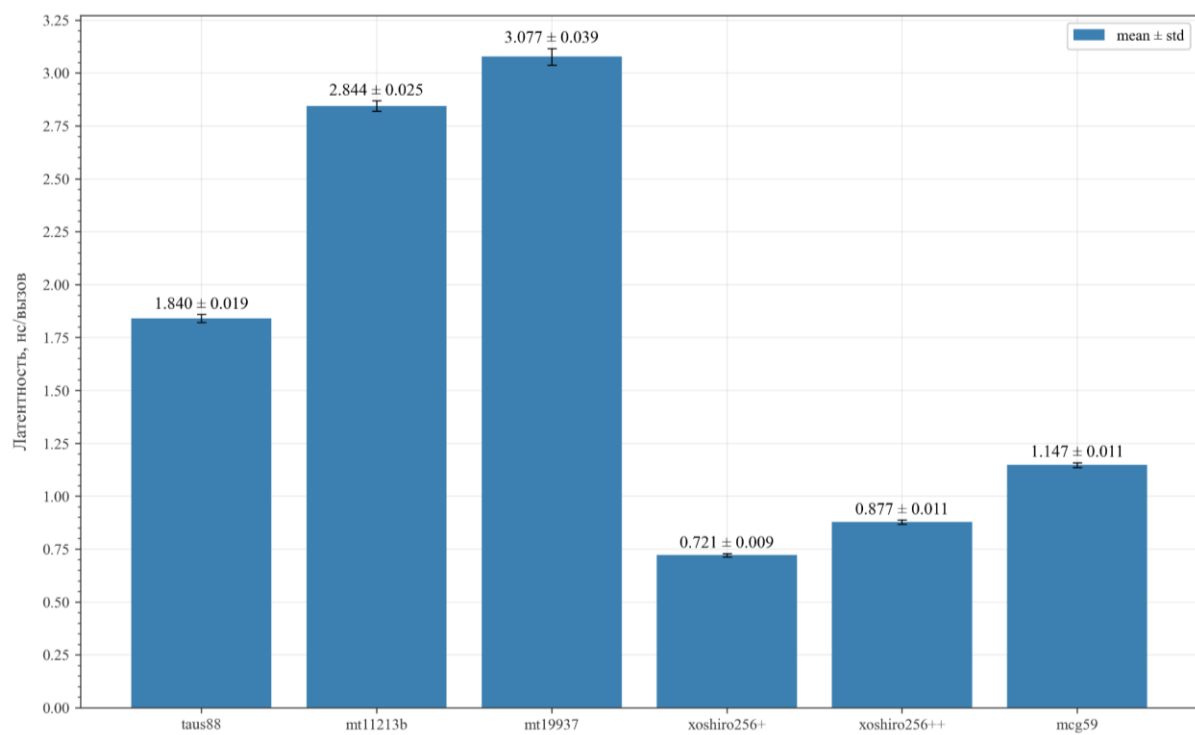


Рис. 1. Латентность ГПСЧ (выборочное среднее)

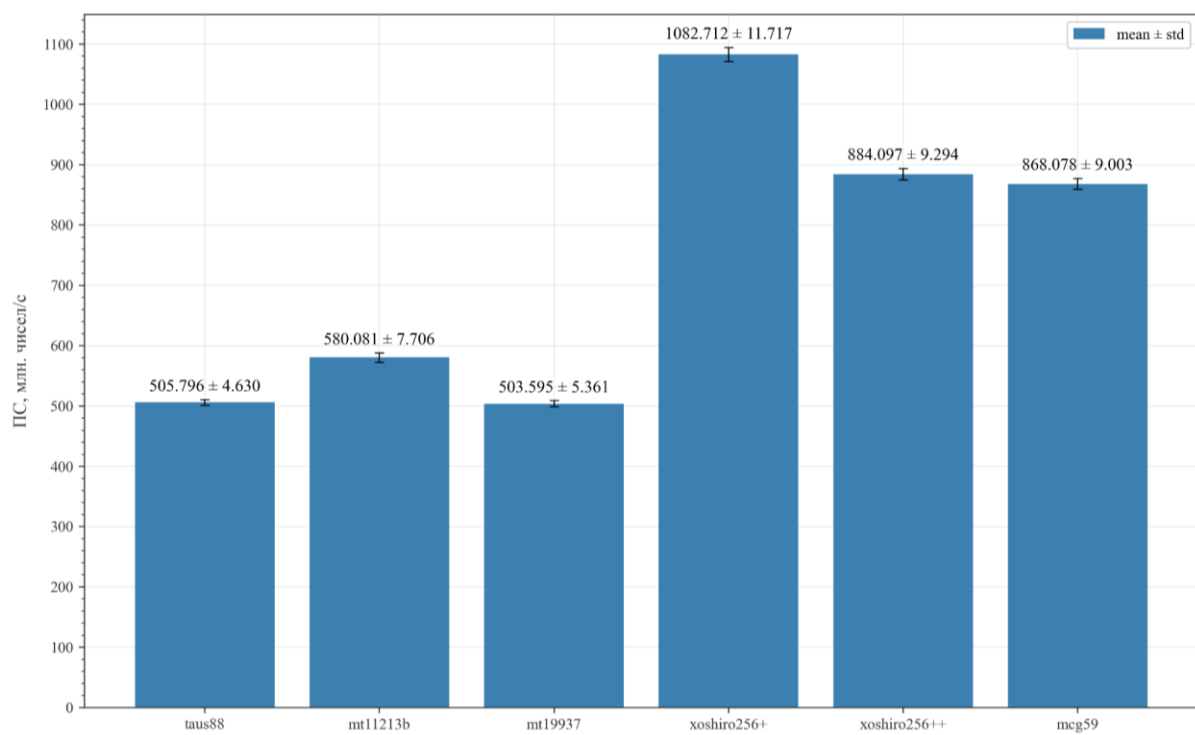


Рис. 2. Пропускная способность ГПСЧ (выборочное среднее)

Результаты бенчмарка показали, что наиболее быстрым в обоих сценариях оказался xorshift-генератор xoshiro256+, что соответствует актуальным данным о производительности ГПСЧ [29].

С точки зрения Монте-Карло моделирования распространения света в значительной степени интересен именно показатель латентности, поскольку сценарий использования ГПСЧ в данном алгоритме соответствует многократным вызовам генератора для получения нескольких случайных чисел на каждый свободный пробег фотона. Согласно полученным данным генератор xoshiro256+ в такой постановке задачи показывает себя лучше других кандидатов.

Интересным кажется результат генератора taus88. При сравнительно низкой латентности он уступает в пропускной способности менее отзывчивому генератору mt11213b, приближаясь к наиболее медленному mt19937. Различия в латентности легко объяснимы: taus88 имеет значительно меньший размер состояния, и сам алгоритм генерации проще (с точки зрения линейной последовательности инструкций ассемблера). Разница же в пропускной способности, предположительно, достигается за счет лучшего параллелизма уровня инструкций (ILP) у генераторов типа Mersenne Twister, что позволяет компилятору оптимизировать конвейерные вычисления внутри итераций цикла заполнения буфера. Данное предположение требует дополнительного исследования.

Несмотря на теоретическое преимущество генераторов с бинарной арифметикой, быстрее многих алгоритмов генерации данного класса оказался msg59. С точки зрения последовательности инструкций его алгоритм крайне прост: для генерации очередного числа msg59 использует одно целочисленное умножение и одно битовое И с константой. В предположении, что константы и состояние генератора не выгружаются в

память, а хранятся в регистрах процессора, для генерации числа требуется всего две инструкции: IMUL и AND. В архитектуре Zen 2 (как и в большинстве современных архитектур) суммарная латентность инструкций IMUL и AND составляет 3+1 такта [30, с. 101-103], что позволяет mcg59 конкурировать в производительности даже с xorshift-генераторами.

Код программы, скрипты для запуска и результаты тестов опубликованы на ресурсе GitHub по адресу <https://github.com/velostone/prng-benchmark>.

3 Оценка статистического качества ГПСЧ

Статистическое качество генераторов оценивалось с помощью тестового набора PractRand [3, 4]. Важно понимать, что такой подход является в некоторой степени качественным, что позволяет ранжировать алгоритмы генерации от худшего к лучшему (по количеству пройденных тестов), но не позволяет дать статистическую оценку прохождения теста (p-value) в случае успеха. Данное исследование не претендует на академическую строгость в этом разделе, отдавая предпочтение более прикладному набору тестов PractRand. Для формального анализа следует обратиться к эталонным наборам из TestU01 [20].

3.1 Алгоритм тестирования

Специфика тестирования набором PractRand предполагает перенаправление потока вывода ГПСЧ в поток ввода исполняемого файла с тестами, который анализирует генерируемые последовательности на предмет соответствия истинной случайности в известных задачах теории вероятностей. Среди тестов присутствуют [31]:

- BCFN (Binary Correlation via Frequency of Numbers), проверяющий наличие линейных корреляций в потоке генерируемых битов с помощью весов Хэмминга. Чувствителен как к локальным (в малой окрестности фиксированного бита), так и к глобальным (далекие друг от друга биты) зависимостям. Данный тест, как правило, не проходят генераторы с выраженной линейностью (LCG, LSFR), в алгоритмах которых не используются скручивание, ротация и иные виды перемешивания;
- BRank, также призванный выявить линейные зависимости, но путем подсчета над $GF(2)$ ранга бинарных матриц, составленных из битов выходной последовательности ГПСЧ. Решает задачу линейной

выразимости результата очередной генерации через систему уже сгенерированных битов. Более чувствителен (по сравнению с BCFN) к линейным генераторам, использующим некоторую степень перемешивания (например, часть алгоритмов класса xorshift);

- Gap-16, составляющий из выходного потока генератора 16-битные числа и измеряющий расстояние (gap) между повторяющимися значениями. От качественного генератора ожидается случайное поведение величины расстояния с распределением, близким к геометрическому (длинные расстояния встречаются реже). Любые устойчивые закономерности в значениях gap или аномальное распределение свидетельствуют о циклической природе генератора (например, MCG) и его низком статистическом качестве;
- FPF (Floating-Point Frequency), интерпретирующий выходные наборы как битовые представления чисел с плавающей точкой (IEEE 754). Специфика интерпретации с применяемым частотным тестом (frequency test) позволяет отследить избыток нулевых битов в отдельных разрядах, локальные шаблоны, повторяющиеся неслучайным образом, и иные дефекты в малых битовых окрестностях. Тест выявляет генераторы с недостаточным перемешиванием, эффект которого проявляется в близких битах.

Для статистического тестирования каждый генератор был обернут в бесконечный цикл с генерацией ПСЧ в поток вывода. Код программы и результаты тестов опубликованы на ресурсе GitHub по адресу <https://github.com/velostone/prng-statistics>.

3.2 Результаты и анализ

В таблице 3 представлены результаты тестирования статистического качества ГПСЧ из таблицы 1 (стартовый пакет включает 210 тестов).

№	Генератор	Объем до остановки	Число пройденных тестов
1	taus88	256 Мбайт	146
2	mt11213b	256 Мбайт	161
3	mt19937	256 Мбайт	161
4	xoshiro256+	256 Мбайт	208
5	xoshiro256++	64+ Гбайт	—
6	mcg59	256 Мбайт	21

Таблица 3. Результаты тестов PractRand

Результаты прохождения PractRand в целом соответствуют теоретическим ожиданиям. Наименее качественным оказался mcg59, прошедший всего 21 тест из стартового пакета. Генератор показал систематические ошибки во всех типах тестов, что говорит о его непригодности для использования в стохастических алгоритмах.

Приемлемый результат продемонстрировали taus88 и генераторы типа Mersenne Twister. Комбинирование и скручивание, используемое в данных алгоритмах, позволило пройти более половины стартовых тестов. Среди прочих тестов генераторы не смогли пройти BCFN, BRank и FPF в разных конфигурациях, что свидетельствует о некоторой степени линейной зависимости выходных последовательностей.

Наивысшее качество продемонстрировал генератор xoshiro256++. Он успешно прошел серию тестовых пакетов, соответствующую генерации 64 Гбайт ПСЧ (тестирование было завершено принудительно). Согласно

актуальным данным xoshiro256++ успешно проходит стресс-тест проверки на зависимости весов Хэмминга (Hamming-Weight Dependencies, [32]) при генерации 1 ПБ псевдослучайных чисел [23]. Учитывая отсутствие систематических ошибок в BigCrush из TestU01, можно говорить об эталонном статистическом качестве xoshiro256++ среди всех известных на сегодняшний день ГПСЧ.

Несколько хуже оказался результат бенчмарк-лидера xoshiro256+. В стартовом пакете генератор неудачно прошел тест BRank в двух конфигурациях: Low1/64 на квадратных матрицах размерностей 384 и 512. Данный тест упаковывает в матрицы первый бит каждой генерируемой строки, отслеживая линейные зависимости младших битов. Автор алгоритма обращает внимание на этот недостаток в своих публикациях и отмечает, что при генерации чисел с плавающей точкой с использованием старших битов статистическое качество xoshiro256+ неотлично от эталонного [29].

Заключение

Анализ результатов бенчмарка и статистического тестирования позволяет рекомендовать xoshiro256+ как наиболее быстрый скалярный ГПСЧ для CPU с эталонным статистическим качеством для генерации чисел с плавающей точкой в задаче Монте-Карло моделирования распространения света в рассеивающих средах.

Литература

[1] Bhattacharjee, K. A Search for Good Pseudo-random Number Generators : Survey and Empirical Studies / K. Bhattacharjee, S. Das // Computer Science Review. – 2022. – Vol. 45. – P. 100471.

[2] Ding, K. Comparison of random number generators in Particle Swarm Optimization algorithm / K. Ding, Y. Tan // 2014 IEEE Congress on Evolutionary Computation (CEC). – 2014. – Pp. 2664-2671.

[3] Doty-Humphrey, C. Practically random: C++ library of statistical tests for rngs / C. Doty-Humphrey. – URL: <https://sourceforge.net/projects/pracrand> (дата обращения: 18.04.2026).

[4] Sleem, L. TestU01 and Pracrand: Tools for a randomness evaluation for famous multimedia ciphers / L. Sleem, R. Couturier // Multimedia Tools and Applications. – 2020. – Vol. 79. – № 33-34. – Pp. 24075-24088.

[5] Кнут, Д. Э. Искусство программирования, том 2. Получисленные алгоритмы / Дональд Э. Кнут ; под ред. Ю. В. Козаченко ; пер. с англ. — 3-е изд. — Москва : Вильямс, 2018. — 832 с. — ISBN 978-5-8459-0081-4.

[6] Comparing pseudo- and quantum-random number generators with Monte Carlo simulations / D. Cirauqui, M. Á. García-March, G. Guigó Corominas [et al.] // APL Quantum. – 2024. – Vol. 1. – № 3. – P. 036125.

[7] Lehmer, D. H. Mathematical Methods in Large-Scale Computing Units / D. H. Lehmer // The Annals of the Computation Laboratory of Harvard University – 1951. – Vol. 26. – Pp. 141-146.

[8] The GNU C Library - ISO Random. – Текст : электронный // GNU. – URL: https://ftp.gnu.org/old-gnu/Manuals/glibc-2.2.3/html_node/libc_386.html (дата обращения: 04.03.2026).

[9] Notes for Intel® oneAPI Math Kernel Library Vector Statistics. – Текст : электронный // Intel. – URL: <https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-vector-statistics-notes/2025-2/mcg59.html> (дата обращения: 22.02.2026).

[10] Vigna, S. Further scramblings of Marsaglia's xorshift generators / S. Vigna // Journal of Computational and Applied Mathematics. – 2017. – Vol. 315. – Pp. 175-181.

[11] Borland, M. Boost Library performance of RNGs / M. Borland – Текст : электронный // Boost C++ Libraries. – URL: https://www.boost.org/doc/libs/latest/doc/html/boost_random/performance.html (дата обращения: 15.02.2026).

[12] Tausworthe, R. Random Numbers Generated by Linear Recurrence Modulo Two / R. Tausworthe // Mathematics of Computation - Math. Comput. – 1965. – Vol. 19. – Pp. 201-209.

[13] Borland, M. Type definition taus88 / M. Borland – Текст : электронный // Boost C++ Libraries. – URL: https://www.boost.org/doc/libs/latest/doc/html/doxygen/headers/taus88_8hpp_1a3822a73532a76cfeca0db9888d676c72.html (дата обращения: 04.03.2026).

[14] L'Ecuyer, P. Maximally equidistributed combined Tausworthe generators / P. L'Ecuyer // Mathematics of Computation. – 1996. – Vol. 65. – № 213. – Pp. 203-213.

[15] Matsumoto, M. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator / M. Matsumoto, T. Nishimura // ACM Trans. Model. Comput. Simul. – 1998. – Vol. 8. – Mersenne twister. – № 1. – Pp. 3-30.

[16] Matsumoto, M. Twisted GFSR generators / M. Matsumoto, Y. Kurita // ACM Trans. Model. Comput. Simul. – 1992. – Vol. 2. – № 3. – Pp. 179-194.

[17] Matsumoto, M. Twisted GFSR generators II / M. Matsumoto, Y. Kurita // ACM Trans. Model. Comput. Simul. – 1994. – Vol. 4. – № 3. – Pp. 254-266.

[18] std::mersenne_twister_engine. – Текст : электронный // cppreference.com. – URL: https://en.cppreference.com/w/cpp/numeric/random/mersenne_twister_engine.html (дата обращения: 11.03.2026).

[19] Borland, M. Type definition mt11213b / M. Borland – Текст : электронный // Boost C++ Libraries. – URL: https://www.boost.org/doc/libs/latest/doc/html/doxygen/headers/mersenne_twister_8hpp_1a29ac4e21d84adda27beb1803a0525ca5.html (дата обращения: 11.03.2026).

[20] L'Ecuyer, P. TestU01: A C library for empirical testing of random number generators / P. L'Ecuyer, R. Simard // ACM Trans. Math. Softw. – 2007. – Vol. 33. – № 4. – Pp. 22:1-22:40.

[21] Marsaglia, G. Xorshift RNGs / G. Marsaglia. – Текст : электронный // Journal of Statistical Software. – 2003. – Vol. 8. – № 14. – URL: <http://www.jstatsoft.org/v08/i14/> (дата обращения: 14.02.2026).

[22] Vigna, S. An Experimental Exploration of Marsaglia's xorshift Generators, Scrambled / S. Vigna // ACM Trans. Math. Softw. – 2016. – Vol. 42. – № 4. – Pp. 30:1-30:23.

[23] Blackman, D. Scrambled Linear Pseudorandom Number Generators / D. Blackman, S. Vigna // ACM Transactions on Mathematical Software. – 2021. – Vol. 47. – № 4. – P. 1-32.

[24] Vigna, S. A PRNG shootout: xoshiro256+ / S. Vigna – Текст : электронный – URL: prng.di.unimi.it/xoshiro256plus.c (дата обращения: 18.04.2026).

[25] Vigna, S. A PRNG shootout: xoshiro256++ / S. Vigna – Текст : электронный – URL: prng.di.unimi.it/xoshiro256plusplus.c (дата обращения: 18.04.2026).

[26] google/benchmark: A microbenchmark support library // google/benchmark. – URL: <https://github.com/google/benchmark> (дата обращения: 17.03.2026).

[27] Huang, H. [FR] Using Random Interleaving to reduce run-to-run variance. / H. Huang – Текст : электронный // google/benchmark. – URL: <https://github.com/google/benchmark/issues/1051> (дата обращения: 18.03.2026).

[28] Borland, M. Chapter 31. Boost.Random / M. Borland – Текст : электронный // Boost C++ Libraries. – URL: https://www.boost.org/doc/libs/latest/doc/html/boost_random.html (дата обращения: 18.03.2026).

[29] Vigna, S. A PRNG shootout / S. Vigna – Текст : электронный – URL: <https://prng.di.unimi.it> (дата обращения: 18.03.2026).

[30] Fog, A. Instruction tables: Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD, and VIA CPUs / A. Fog – Текст : электронный – URL: https://www.agner.org/optimize/instruction_tables.pdf (дата обращения: 18.03.2026).

[31] Doty-Humphrey, C. Practically random: C++ library of statistical tests for rngs, Tests_engines.txt / C. Doty-Humphrey. – URL: prcrand.sourceforge.net/Tests_engines.txt (дата обращения: 18.04.2026).

[32] Blackman, D. A New Test for Hamming-Weight Dependencies / D. Blackman, S. Vigna // ACM Transactions on Modeling and Computer Simulation. – 2022. – Vol. 32. – № 3. – P. 1-13.